Progettazione Modulo Didattico Pratico (6–8 Ore): Git, Unsloth Studio e OpenCode per ITS MAKER Bologna

Percorso: Digitalizzazione Industriale e AI Open Source per la Manifattura

Destinatari: 20–24 studenti ITS MAKER Bologna (background prevalentemente da istituti professionali)

Approccio: 80% Laboratorio Pratico Hands-On / 20% Inquadramento Concettuale

Infrastruttura Target: Aula VMware con risorse hardware ridotte (CPU condivisa, RAM limitata, assenza di GPU dedicata, rete condivisa)

---

1. Strategia di Sopravvivenza Tecnica per l'Aula VMware (Pre-requisiti e Piano B)

Con 20–24 macchine virtuali (VM) che condividono lo stesso host ESXi/VMware e la stessa rete scolastica, **il download simultaneo di modelli o pacchetti durante la lezione bloccherà l'aula in 3 minuti**. Tutta la preparazione pesante deve risiedere nella *Golden Image* (snapshot base della VM preparata prima del corso).

1.1 Checklist Preparazione "Golden Image" VMware (Da eseguire prima della lezione)

- ☐ **Risorse minime per singola VM:** almeno 2 vCPU (consigliate 4 vCPU shared), 6–8 GB RAM, 20 GB disco libero.
- ☐ **Pre-installazione binari (Zero-Install in aula):**
- `git` + configurazione globale di fallback già pronta o script di benvenuto.
- **Unsloth Studio** installato (`curl -fsSL https://unsloth.ai/install.sh | sh` su Linux/WSL oppure installer Desktop Windows).
- **OpenCode** (CLI + Desktop App da `opencode.ai/download`).
- Browser leggero (Chromium / Firefox) e editor leggero (VS Code senza estensioni pesanti o Notepad++).
- ☐ **Pre-popolamento Cache Modelli GGUF (Fondamentale):**
- Scaricare nella Golden Image i file `.gguf` direttamente nella cartella standard letta da Unsloth Studio:
- **Linux/WSL:** `~/.cache/huggingface/hub/`
- **Windows:** `C:\Users\<utente>\.cache\huggingface\hub\`
- Impostare la variabile d'ambiente `HF_HUB_OFFLINE=1` (attivabile con uno script sul Desktop `attiva_modalita_offline.bat/.sh`) se la rete d'aula cade.
- ☐ **Cache Locale dei Repository Git Didattici:**
- Creare nella cartella `~/officina-template/` una copia già clonata dei repository didattici (così se GitHub blocca 24 IP natting uguali, gli studenti clonano da cartella locale o dal PC del docente via LAN: `git clone http://192.168.x.100:8000/template-maker.git`).
- ☐ **Snapshot VMware "STATO_ZERO_PULITO":**
- Creare uno snapshot della VM pronta all'uso per ripristinare in 30 secondi la postazione di uno studente che cancella l'ambiente per errore.

1.2 Selezione Modelli Ultra-Quantizzati per Unsloth Studio (Zero GPU / Solo CPU)

Per far toccare con mano agli studenti il compromesso tra **compressione (quantizzazione)**, **velocità (token/s su CPU)** e **intelligenza**, caricheremo nella Golden Image esattamente 4 file GGUF (totale ~3.1 GB su disco, nessun singolo file > 1.8 GB):

Modello (Repository Unsloth)	Parametri (Miliardi)	Quantizzazione	Dimensione File (MB)	RAM Richiesta (MB)	Velocità Stima CPU VM (tok/s)	Ruolo Didattico in Aula
`unsloth/Qwen3-0.6B-GGUF`	0.6	`Q4_K_M`	395	750	28	Test velocità estrema: risponde istantaneamente su CPU ma sbaglia logiche complesse
`unsloth/Llama-3.2-1B-Instruct-GGUF`	1.2	`UD-IQ2_M` (1.58–2 bit)	450	850	22	Dimostrazione iper-compressione: mostra gli effetti di una quantizzazione estrema (allucinazioni su calcoli)
`unsloth/Qwen2.5-Coder-1.5B-Instruct-GGUF`	1.5	`Q4_K_M` (4-bit)	985	1450	14	**Cavallo di battaglia VM locale**: ottimo in italiano e capace di generare HTML/JS/Python pulito
`unsloth/Qwen2.5-Coder-3B-Instruct-GGUF`	3.0	`UD-Q3_K_M` (3-bit Dynamic)	1520	2200	7	Modello "pesante" per la VM: codice più raffinato, ma fa sentire la fatica della CPU condivisa

1.3 Architettura a 3 Livelli per il Backend di OpenCode (Piano A, Piano B, Piano C)

Durante la fase di **Vibe Coding**, se tutti i 24 studenti lanciano contemporaneamente una generazione di codice da 500 righe sulla CPU virtualizzata, l'host VMware potrebbe andare in *CPU thrashing* (100% utilizzo host). Prepariamo tre modalità intercambiabili in 30 secondi:

- **Piano A – Esecuzione 100% Locale sulla VM (Default per capire il motore):**
- Lo studente ha `Qwen2.5-Coder-1.5B-Instruct-GGUF` caricato su Unsloth Studio (`localhost:8888`).
- Lancia da terminale nella cartella di progetto:

```
unsloth start opencode
```

- Unsloth crea in automatico la chiave `sk-unsloth-...` e collega OpenCode al modello locale.
- **Piano B – Server Unsloth Condiviso sul PC del Docente in LAN (Se le VM rallentano):**
- Il docente avvia Unsloth sul proprio portatile (più potente o dotato di GPU) esponendo l'API sulla rete locale dell'aula:

```
unsloth run \
  --model unsloth/Qwen2.5-Coder-7B-Instruct-GGUF \
  -H 0.0.0.0 \
  --disable-tools \
  -p 8888
```

- **Nota tecnica critica:** il flag `--disable-tools` è **obbligatorio** quando si usa `unsloth run` come server per OpenCode, altrimenti Unsloth intercetta le chiamate agli strumenti lato server e OpenCode risponde in chat senza mai creare o modificare i file!
- Gli studenti puntano OpenCode a `http://<IP_DOCENTE>:8888/v1/`. Zero carico CPU sulle VM degli studenti.
- **Piano C – Fallback API Cloud Gratuita (OpenRouter / Groq Free Tier):**
- Se sia l'host VMware sia la LAN hanno problemi di latenza, il docente fornisce una API Key didattica pre-configurata con modelli gratuiti (`qwen/qwen-2.5-coder-32b-instruct:free` su OpenRouter). Consuma pochi KB di banda (solo testo JSON) e garantisce velocità istantanea.

---

2. Filo Conduttore Unificato (Il Workflow Git + Unsloth + OpenCode)

Per studenti di istituto professionale (futuri tecnici dell'automazione, manutenzione 4.0 e gestione produzione), evitiamo astrazioni informatiche. Usiamo la **Metafora dell'Officina Meccanica Digitale**:

1. **Git e GitHub = Il Magazzino Ricambi e il Pulsante di Ripristino Macchina:**

- *Open Source / GitHub:* Non si costruisce mai una vite da zero; si entra a magazzino, si legge la scheda tecnica (`README.md`) e si preleva un semilavorato (`git clone`).
- *Git locale:* Quando modifichi i parametri di un controllo numerico o di un software, fai un punto di salvataggio (`git commit`). Se l'ultima modifica rompe tutto, premi il fungo di emergenza e torni esattamente all'ultimo pezzo buono (`git restore` / `git reset`).

1. **Unsloth Studio = Il Banco Prova Motori:**

- Prima di montare un motore AI su una macchina utensile o su un tablet di reparto senza internet, dobbiamo provarlo al banco. Quanto pesa? Quanta RAM consuma? Se lo comprimiamo a 2-bit o 4-bit (quantizzazione GGUF), gira anche su un PC industriale economico senza scheda video?

1. **OpenCode = L'Operatore Robotico di "Vibe Coding":**

- Tu sei il **Capo Reparto / Responsabile Ufficio Tecnico**: conosci il problema reale (es. le schede di manutenzione cartacee si perdono o sono sporche di grasso). OpenCode è l'esecutore che scrive il codice. Ma poiché l'AI può sbagliare ("allucinare"), **la regola d'oro dell'officina è: non chiedere mai a OpenCode il passo successivo se prima non hai salvato su Git il passo precedente funzionante**.

Schema del Flusso Operativo Unificato

1. **FASE 1 – MAGAZZINO OPEN SOURCE (Git):** Esplora Repository & `README.md` → `git clone template-

`base`

2. **FASE 2 – BANCO PROVA AI (Unsloth Studio):** Test GGUF 1.58-bit vs 4-bit su CPU → Scelta Modello Locale (`Qwen2.5-Coder-1.5B`)
3. **FASE 3 – CICLO VIBE CODING SICURO (OpenCode + Git):**

- Invio Prompt `v1` a OpenCode → Verifica nel Browser
- **Se funziona:** `git add . && git commit -m "v1 ok"` → Passa a Prompt `v2`
- **Se l'AI rompe il file:** `git restore .` (Fungo di Emergenza) → Riformula il prompt in modo chirurgico

---

3. Cronoprogramma Modulare (6 Ore Core + 2 Ore Estensione)

La tabella seguente scandisce tempi, taglio 6h vs 8h e ripartizione tra teoria (20%) e pratica (80%).

Blocco	Durata (min)	Minuti Teoria	Minuti Pratica	Titolo Modulo	Stato (6h vs 8h)	Cosa fa il Docente (Al Proiettore)	Cosa fanno gli Studenti (Sulla VM)
B1	60	15	45	**L'Ecosistema Open Source e Git come "Salvataggio Partita"**	**CORE (Ora 1)**	Mostra come cercare tool industriali su GitHub, leggere un `README.md` per capire se un progetto è vivo o abbandonato, e introduce i 4 comandi base di Git con la metafora dei checkpoint dei videogiochi.	Esplorano 2 repository preparati, clonano il template `officina-base`, modificano un file di configurazione reparto, eseguono `git status`, `git add`, `git commit`, poi rompono intenzionalmente il file e lo salvano con `git restore`.
B2	60	15	45	**Dentro il Motore AI: Unsloth Studio e il Peso dei Modelli (GGUF)**	**CORE (Ora 2)**	Avvia `unsloth studio -p 8888`, spiega senza formule cos'è un parametro (0.6B vs 3B) e cos'è la quantizzazione (da 16-bit a 4-bit/1.58-bit = comprimere un disegno CAD in un PDF leggero). Mostra i badge `RECOMMENDED`, `TIGHT`, `OOM`.	Avviano Unsloth Studio sulla VM, aprono il *Task Manager / Monitor di Sistema* accanto al browser e caricano i modelli pre-scaricati. Annotano in una scheda tecnica i MB di RAM occupati e i token/s generati.
B3	60	10	50	**Stress Test Industriale: Velocità vs Allucinazioni**	**CORE (Ora 3)**	Mostra la **Model Arena** (confronto fianco a fianco) di Unsloth Studio.	Eseguono 3 test pratici nella Model Arena: 1) Estrazione dati da una bolla di consegna

Blocco	Durata (min)	Minuti Teoria	Minuti Pratica	Titolo Modulo	Stato (6h vs 8h)	Cosa fa il Docente (Al Proiettore)	Cosa fanno gli Studenti (Sulla VM)
				nella Model Arena		Sottopone una nota di fermo macchina reale a due modelli diversi (uno 1B ultra-compresso 2-bit e uno 1.5B a 4-bit) mostrando dove il modello piccolo inventa dati.	testuale; 2) Traduzione tecnica IT->EN per manuale macchina; 3) Generazione di una tabella HTML. Decidono quale modello usare come "cervello" per OpenCode.
B4	60	15	45	**Dal Motore al Braccio Operativo: Collegare Unsloth a OpenCode**	**CORE (Ora 4)**	Spiega cos'è un agente di *Vibe Coding*: differenza tra "chattare con l'AI e fare copia-incolla" vs "dare a OpenCode il permesso di leggere e scrivere i file nella cartella". Mostra `unsloth start opencode` (e attiva il Piano B LAN se le VM faticano).	Collegano OpenCode a Unsloth Studio, scelgono uno dei 5 progetti del Catalogo Digitalizzazione e lanciano il **Prompt v1**. Aprono il file generato nel browser e fanno il primo **Commit Git ("v1 funzionante")**.
B5	60	10	50	**Vibe Coding Iterativo & Il "Disastro Controllato" (v2 -> Bug -> Rollback)**	**CORE (Ora 5)**	Guida l'iterazione **v2** (aggiunta filtri/calcoli KPI). Poi simula in diretta l'errore tipico: un prompt vago che distrugge la grafica o blocca i pulsanti. Mostra come usare `git diff` per vedere il danno e `git restore .` per tornare in 2 secondi alla v1/v2 funzionante.	Eseguono il **Prompt v2**, verificano e fanno commit. Poi lanciano il **Prompt v3 (Trappola/Fix)**: sperimentano il blocco, usano Git per annullare il danno o correggere con un prompt chirurgico, e consolidano la versione finale.
B6	60	10	50	**Collaudo Finale, Documentazione README e Consegna Repository**	**CORE (Ora 6)**	Mostra come far scrivere a OpenCode un `README.md` professionale (con istruzioni d'uso per i colleghi di reparto) e	Chiedono a OpenCode di generare il `README.md` del proprio tool, eseguono l'ultimo commit (e `git push` se previsto) e fanno una

Blocco	Durata (min)	Minuti Teoria	Minuti Pratica	Titolo Modulo	Stato (6h vs 8h)	Cosa fa il Docente (Al Proiettore)	Cosa fanno gli Studenti (Sulla VM)
						come verificare il registro storico del lavoro con `git log --oneline`.	demo lampo di 2 minuti al compagno di banco ("collaudo incrociato").
B7	60	10	50	**[ESTENSIONE 8h] Dati Sporchi Reali: Da File CSV/Log Macchina a Tool su Misura**	**OPZIONALE (Ora 7)**	Fornisce un file `.csv` reale "sporco" (separatori misti `;` e `,`, righe vuote, codici articolo errati esportati da un vecchio gestionale/PLC) e mostra come darlo in pasto a Unsloth Chat (upload file) e OpenCode.	Usano OpenCode per creare o estendere il proprio applicativo affinché importi il file CSV sporco dell'ufficio tecnico, segnali in rosso le righe corrotte e produca un report pulito scaricabile.
B8	60	10	50	**[ESTENSIONE 8h] Personalizzazione Istruzioni Agente (`AGENTS.md` / System Prompt) & Sfida di Reparto**	**OPZIONALE (Ora 8)**	Spiega come istruire stabilmente il modello (System Prompt su Unsloth Studio o file `AGENTS.md` nella cartella per OpenCode) con le regole aziendali (colori standard sicurezza ISO, lingua italiana, zero librerie esterne).	Creano il file di regole aziendali del proprio reparto, chiedono un'ultima personalizzazione grafica/funzionale a OpenCode rispettando lo standard, salvano su Git e presentano il tool alla classe.

> **Regola di Taglio Rapido (6h vs 8h):**
>
> - **Formato 6 Ore:** Eseguire i blocchi **B1** → **B6**. Nei progetti di Vibe Coding (B4–B5) usare dati di esempio direttamente integrati nella pagina HTML/JS senza caricamento di file esterni complessi.
>
> - **Formato 8 Ore:** Includere **B7 e B8**, introducendo la pulizia di file CSV/log macchina reali e la configurazione di regole aziendali permanenti (`AGENTS.md`).

---

4. Catalogo di 5 Idee Pratiche di "Vibe Coding" con OpenCode (Tema Digitalizzazione ITS MAKER)

Tutti i 5 progetti sono progettati per:

- **Zero dipendenze pesanti:** un singolo file `index.html` (con CSS e JavaScript inclusi) o uno script Python standard. Si aprono con un doppio clic nel browser della VM anche **senza connessione internet**.
- **Alto impatto visivo immediato:** colori da bordo macchina (verde/giallo/rosso), tabelle interattive, contatori e pulsanti grandi adatti a un monitor touch di reparto.

Progetto 1: Scheda Manutenzione Preventiva & Checklist 5S Bordo Macchina

- **Problema reale:** In officina le schede di controllo giornaliero (livello olio, pulizia filtri, verifica protezioni, pressione aria) sono fogli di carta appesi alla macchina che si macchiano, si perdono e nessuno ricopia su Excel.
- **Risultato:** Web-app in un unico file `manutenzione.html` con spunte interattive, barra di completamento, salvataggio automatico nel browser (`localStorage`) ed esportazione del report di fine turno in CSV.

Sequenza Operativa Prompt + Git:

1. **Prompt v1 (Base funzionante):**

```
   Crea in un singolo file chiamato manutenzione.html (con CSS e JavaScript inclusi nello stesso file, senza librerie esterne) una scheda digitale di Manutenzione Giornaliera per un Centro di Lavoro CNC "Mazak-04".

   Inserisci:

   - Campi in alto per: Nome Operatore, Turno (Mattina/Pomeriggio/Notte) e Data odierna automatica.

   - Una lista di 6 controlli pratici (Livello olio guide, Pressione pneumatica 6 bar, Pulizia vasca trucioli, Integrità fungo emergenza, Controllo refrigerante, Pulizia fotocellule) con pulsanti grandi "OK" (verde), "ANOMALIA" (rosso) e campo note per ogni voce.

   - Usa uno stile scuro industriale ad alto contrasto leggibile su monitor di reparto.
```

1. **Checkpoint Git 1:**

```
   git add manutenzione.html

   git commit -m "v1: scheda manutenzione base CNC funzionante"
```

1. **Prompt v2 (Aggiunta funzionalità e salvataggio):**

```
   Modifica manutenzione.html aggiungendo:

   1. Una barra di avanzamento in alto che mostra la percentuale di controlli completati.

   2. Un avviso lampeggiante rosso in cima alla pagina se almeno un controllo è segnato come "ANOMALIA" ("ATTENZIONE: MACCHINA FERMA - CHIAMARE MANUTENZIONE").

   3. Un pulsante "Salva ed Esporta CSV Fine Turno" che salva i dati nel localStorage del browser (così se ricarico la pagina non perdo le spunte) e scarica un file .csv con l'esito dei controlli.
```

1. **Checkpoint Git 2:**

```
git add manutenzione.html

git commit -m "v2: aggiunta barra avanzamento, allarme anomalia ed export CSV"
```

1. **Prompt v3 (Fix di un difetto reale + Verifica):**

```
C'è un problema operativo: se un operatore clicca "Salva ed Esporta CSV Fine Turno" senza aver inserito il Nome Operatore o senza aver controllato tutte le 6 voci, il file viene scaricato comunque incompleto.

Aggiungi una validazione che blocca l'esportazione, evidenzia in giallo i campi mancanti e mostra un messaggio chiaro con l'elenco dei controlli ancora da eseguire. Aggiungi anche un pulsante "Inizia Nuovo Turno" con conferma di sicurezza per azzerare la scheda.
```

1. **Checkpoint Git 3:**

```
git add manutenzione.html

git commit -m "v3: validazione campi obbligatori e reset fine turno"
```

Progetto 2: Validatore e Convertitore Distinta Base (BOM Inspector per Ufficio Tecnico)

- **Problema reale:** L'ufficio progettazione esporta dal CAD le distinte base (BOM - *Bill of Materials*) in CSV, ma prima di passarle agli acquisti o all'officina ci sono codici duplicati, quantità a zero o materiali mancanti che causano ordini sbagliati.
- **Risultato:** Applicazione `bom_inspector.html` dove si incolla o carica una distinta base CSV (con pulsante "Carica Dati di Esempio" già pronto), il programma trova gli errori automaticamente, calcola i costi totali e permette di scaricare la distinta corretta.

Sequenza Operativa Prompt + Git:

1. **Prompt v1 (Base funzionante):**

```
Crea un singolo file bom_inspector.html (HTML, CSS e JS tutto incluso, zero dipendenze esterne) per l'Ufficio Tecnico di un'azienda meccanica.

La pagina deve avere:

- Un'area di testo dove incollare una Distinta Base in formato CSV (colonne: Codice_Articolo, Descrizione, Materiale, Quantita, Costo_Unitario).

- Un pulsante "Carica Distinta di Prova (con errori)" che riempie automaticamente l'area con 10 componenti meccanici (carpenteria, viti M8, cuscinetti SKF, alberi rettificati) includendo di proposito 2 codici duplicati, 1 quantità uguale a 0 e 1 riga senza materiale.

- Un pulsante "Analizza Distinta" che mostra i dati in una tabella ordinata.
```

1. **Checkpoint Git 1:**

```
git add bom_inspector.html

git commit -m "v1: lettore distinta base CSV con dati di prova"
```

1. **Prompt v2 (Controllo qualità automatico e calcolo costi):**

```
   Aggiorna bom_inspector.html per aggiungere il controllo qualità automatico della distinta base:

   - Evidenzia in rosso nella tabella le righe con Quantita <= 0 o Materiale vuoto.

   - Evidenzia in arancione le righe che hanno lo stesso Codice_Articolo ripetuto più volte.

   - Mostra in alto 3 riquadri riepilogativi (KPI): Numero Totale Componenti, Numero Errori Trovati, Costo Totale Commessa (somma di Quantita * Costo_Unitario).
```

1. **Checkpoint Git 2:**

```
   git add bom_inspector.html

   git commit -m "v2: validazione errori distinta e calcolo costo commessa"
```

1. **Prompt v3 (Fix ed eliminazione automatica duplicati):**

```
   Quando ci sono due righe con lo stesso Codice_Articolo, l'ufficio acquisti vuole unificarle sommando le quantità invece di ordinarle due volte, e c'è un bug se il Costo_Unitario usa la virgola invece del punto decimale (es. "12,50" dà NaN).

   1. Correggi il calcolo in modo che accetti sia la virgola sia il punto nei prezzi.

   2. Aggiungi un pulsante "Ripara Automaticamente Distinta" che unisce i codici duplicati sommando le quantità, rimuove le righe con quantità 0 e permette di scaricare il CSV pulito.
```

1. **Checkpoint Git 3:**

```
   git add bom_inspector.html

   git commit -m "v3: fix virgola decimale e unione automatica codici duplicati"
```

Progetto 3: Dashboard Controllo Scarti & Semaforo Qualità di Reparto

- **Problema reale:** A bordo linea (stampaggio plastico, lavorazioni CNC o packaging automatico) gli operatori segnano i pezzi scartati con delle crocette su un foglio; a fine giornata nessuno sa subito quale difetto sta causando più perdite.
- **Risultato:** Cruscotto `controllo_scarti.html` stile Andon (semaforo di reparto) con pulsanti rapidi `+1 Pezzo Buono` e `+1 Scarto` divisi per causale (Bava, Fuori Tolleranza, Graffiato, Errore Montaggio), calcolo % scarto in tempo reale e grafico a barre delle causali.

Sequenza Operativa Prompt + Git:

1. **Prompt v1 (Base funzionante):**

```
   Crea in un singolo file controllo_scarti.html (HTML+CSS+JS autonomo) un contatore digitale per il controllo qualità a bordo macchina.

   Inserisci:
```

```
   - Selezione Codice Lotto e Obiettivo Pezzi del turno (es. 500 pezzi).

   - Un pulsante grande verde "+1 PEZZO CONFORME (+10 con tasto rapido)" e 4 pulsanti grandi rossi per registrare gli scarti secondo la causale: "Fuori Tolleranza", "Difetto Superficiale", "Bava di Lavorazione", "Danno Materiale Grezzo".

   - Tre contatori giganti: Pezzi Buoni, Pezzi Scartati, Percentuale di Scarto (%).
```

1. **Checkpoint Git 1:**

```
   git add controllo_scarti.html

   git commit -m "v1: contatore pezzi conformi e scarti per causale"
```

1. **Prompt v2 (Semaforo Andon e Grafico Pareto delle causali):**

```
   Arricchisci controllo_scarti.html con:

   1. Un semaforo industriale in alto che cambia colore: VERDE se lo scarto è sotto il 3%, GIALLO tra 3% e 5%, ROSSO lampeggiante sopra il 5%.

   2. Un grafico a barre orizzontali realizzato in puro CSS/JS (senza librerie esterne) che ordina le 4 causali di scarto dalla più frequente alla meno frequente (analisi di Pareto), così il capoturno vede subito il problema principale.
```

1. **Checkpoint Git 2:**

```
   git add controllo_scarti.html

   git commit -m "v2: semaforo soglie scarto e grafico Pareto causali"
```

1. **Prompt v3 (Fix errore operatore "Annulla Ultimo Click" + Storico):**

```
   Problema pratico in reparto: con i guanti da lavoro l'operatore spesso preme per sbaglio due volte un pulsante di scarto e falsa la percentuale.

   Aggiungi:

   1. Un pulsante "Annulla Ultima Registrazione (Undo)" che cancella l'ultimo inserimento fatto e ricalcola subito percentuale, semaforo e grafico.

   2. Un registro in basso delle ultime 10 pressioni con l'orario esatto (HH:MM:SS).
```

1. **Checkpoint Git 3:**

```
   git add controllo_scarti.html

   git commit -m "v3: tasto annulla ultimo inserimento e log orario"
```

Progetto 4: Calcolatore Tempi Ciclo, Takt Time e Bilanciamento Linea d'Assemblaggio

- **Problema reale:** Chi lavora in Tempi e Metodi o nell'industrializzazione prodotto deve capire se una linea a 4 postazioni riesce a soddisfare la domanda del cliente o quale postazione fa da "collo di bottiglia"

(*bottleneck*).

- **Risultato:** Simulatore visivo `tempi_ciclo.html` con cronometro integrato per rilievo tempi, calcolo automatico del *Takt Time* e grafico delle postazioni con evidenziazione automatica del collo di bottiglia.

Sequenza Operativa Prompt + Git:

1. **Prompt v1 (Base funzionante):**

```
   Crea in un singolo file tempi_ciclo.html (HTML+CSS+JS senza librerie esterne) un Calcolatore di Takt Time e Bilanciamento Linea per una linea di assemblaggio meccatronico a 4 stazioni (Stazione 1: Pre-montaggio, Stazione 2: Cablaggio, Stazione 3: Collaudo Elettrico, Stazione 4: Imballaggio).

   Inserisci:

   - Input per: Minuti lavorativi per turno (default 450 min) e Pezzi richiesti dal cliente per turno (default 300 pezzi).

   - Calcolo automatico del Takt Time in secondi per pezzo.

   - Input del tempo ciclo (in secondi) per ciascuna delle 4 stazioni e visualizzazione a barre verticali confrontate con la linea rossa del Takt Time.
```

1. **Checkpoint Git 1:**

```
   git add tempi_ciclo.html

   git commit -m "v1: calcolo takt time e confronto 4 stazioni"
```

1. **Prompt v2 (Individuazione Collo di Bottiglia e Capacità Reale):**

```
   Modifica tempi_ciclo.html aggiungendo:

   1. L'identificazione automatica della stazione "COLLO DI BOTTIGLIA" (quella col tempo più alto): colorala in rosso se supera il Takt Time, in arancione se è la più lenta ma sotto il Takt Time.

   2. Il calcolo della "Produzione Massima Reale a Fine Turno" basata sul collo di bottiglia e il semaforo "OBIETTIVO CLIENTE RAGGIUNGIBILE: SÌ / NO".
```

1. **Checkpoint Git 2:**

```
   git add tempi_ciclo.html

   git commit -m "v2: rilevamento collo di bottiglia e capacita produttiva reale"
```

1. **Prompt v3 (Cronometro Rilievo Tempi & Simulazione Spostamento Mansioni):**

```
   Aggiungi due funzioni pratiche per il tecnico Tempi e Metodi:

   1. Un mini-cronometro (Start / Stop / Invia a Stazione selezionata) direttamente nella pagina per cronometrare dal vivo un'operazione e inserire il valore misurato nella stazione scelta.

   2. Correggi il caso in cui l'utente inserisca 0 pezzi richiesti o valori negativi evitando
```

```
che compaia "Infinity" o "NaN" nei risultati.
```

1. **Checkpoint Git 3:**

```
   git add tempi_ciclo.html

   git commit -m "v3: cronometro integrato e protezione divisione per zero"
```

Progetto 5: Generatore Etichette Pallet & Tracciabilità Lotto Magazzino

- **Problema reale:** A fine linea bisogna attaccare sul bancale o sul cassone metallico una scheda identificativa chiara (Codice Articolo, Lotto, Data, Quantità, Destinazione/Reparto successivo) ma i fogli scritti a pennarello sono illeggibili dai carrellisti.
- **Risultato:** Web-app `etichette_lotto.html` che genera istantaneamente un'etichetta industriale standardizzata pronta per la stampa (con codice identificativo univoco e griglia visiva stile QR/codice a barre disegnato in SVG puro offline) e mantiene lo storico dei bancali creati in giornata.

Sequenza Operativa Prompt + Git:

1. **Prompt v1 (Base funzionante):**

```
   Crea un singolo file etichette_lotto.html (HTML+CSS+JS autonomo, zero internet) per
generare schede di tracciabilità bancali in officina.

   La schermata deve essere divisa in due:

   - A sinistra il modulo di inserimento: Codice Articolo, Descrizione Pezzo, Numero Lotto
(generabile anche in automatico nel formato LOT-AAAAMMGG-XX), Quantità Pezzi nel cassone,
Operatore e Reparto Destinazione (Trattamento Termico, Verniciatura, Montaggio, Spedizione).

   - A destra l'anteprima in tempo reale dell'etichetta industriale con bordi neri spessi,
caratteri giganti leggibili dal muletto a 3 metri di distanza e un codice a barre SVG
generato direttamente in JavaScript a partire dal Numero Lotto.
```

1. **Checkpoint Git 1:**

```
   git add etichette_lotto.html

   git commit -m "v1: generatore etichetta bancale con codice a barre SVG"
```

1. **Prompt v2 (Bollino Stato Qualità e Registro Bancali Giornata):**

```
   Aggiungi a etichette_lotto.html:

   1. Un selettore "Stato Qualità Lotto" con 3 opzioni che cambiano l'intestazione
dell'etichetta: "CONFORME - LIBERO PASSAGGIO" (Verde), "IN ATTESA COLLAUDO" (Giallo),
"BLOCCATO / NON CONFORME" (Rosso con banda diagonale).

   2. Un pulsante "Registra Bancale e Stampa" che aggiunge il bancale a una tabella "Storico
Bancali Prodotti Oggi" in fondo alla pagina (salvata in localStorage) e apre la finestra di
stampa (@media print deve stampare SOLO l'etichetta a pagina intera, nascondendo il form).
```

1. **Checkpoint Git 2:**

```
   git add etichette_lotto.html

   git commit -m "v2: stato qualita visivo, storico bancali e layout di stampa"
```

1. **Prompt v3 (Fix contatore progressivo collo e ristampa):**

```
   Nella tabella "Storico Bancali Prodotti Oggi" manca la possibilità di recuperare un'etichetta se si è inceppata la stampante, e il numero progressivo del lotto non avanza da solo dopo ogni registrazione.

   1. Fai in modo che cliccando "Registra Bancale" il contatore finale del lotto aumenti automaticamente di 1 (es. da LOT-20261018-01 a LOT-20261018-02).

   2. Aggiungi su ogni riga dello storico un pulsante "Ricarica Etichetta" che riporta tutti i dati di quel bancale nell'anteprima per ristamparla.
```

1. **Checkpoint Git 3:**

```
   git add etichette_lotto.html

   git commit -m "v3: incremento automatico lotto e pulsante ristampa dallo storico"
```

---

5. Tracce delle Esercitazioni Guidate & I 5 Errori più Frequenti in Aula

5.1 Scheda Operativa Rapida per gli Studenti (I 6 Comandi Salvavita)

```
# 1. SCARICA UN PROGETTO ESISTENTE (Preleva dal magazzino)

git clone <url-o-percorso-template>

cd <nome-cartella>


# 2. CONTROLLA COSA È CAMBIATO (Ispeziona il banco di lavoro)

git status


# 3. METTI IN CASSAFORTE UNA VERSIONE FUNZIONANTE (Salva partita)

git add .

git commit -m "v1: descrizione breve di cosa funziona"


# 4. FUNGO DI EMERGENZA (L'AI ha rotto il file! Torna all'ultimo salvataggio buono)

git restore .
```

```
# 5. AVVIA IL BANCO PROVA MOTORI AI (Interfaccia Web Unsloth Studio)
unsloth studio -p 8888

# 6. AVVIA L'OPERATORE DI VIBE CODING COLLEGATO AL MODELLO LOCALE
unsloth start opencode
```

5.2 Traccia Esercitazione 1: "Il Fungo di Emergenza Git" (30 min - Blocco B1)

- **Docente al Proiettore:**

1. Mostra la cartella `officina-base` contenente un piccolo file `parametri_macchina.html` (una tabellina con temperature e velocità mandrino).
2. Fa una modifica corretta, la mostra nel browser, poi esegue `git add .` e `git commit -m "parametri base collaudati"`.
3. Cancella metà del file simulando un errore umano o un crash, aggiorna il browser (pagina distrutta!) e mostra il panico tipico di chi fa `File_definitivo_ultimo_v4_vero.html`.
4. Digita `git restore .`, aggiorna il browser e mostra che la pagina è tornata intatta in mezzo secondo.

- **Studenti sulla VM:**

1. Clonano il repository locale `git clone ~/officina-template/officina-base`.
2. Aprono `parametri_macchina.html`, inseriscono il proprio Nome e Cognome come "Responsabile Macchina" e cambiano la velocità mandrino a `4500 RPM`.
3. Salvano su Git (`git add .` + `git commit -m "inserito responsabile e RPM"`).
4. Aprono il file, cancellano tutto il contenuto e salvano il file vuoto. Verificano nel browser che la pagina è bianca.
5. Usano `git status` e `git restore .` per recuperare istantaneamente il lavoro.

5.3 Traccia Esercitazione 2: "Il Banco Prova GGUF su Unsloth Studio" (45 min - Blocchi B2/B3)

- **Docente al Proiettore:**

1. Apre `http://localhost:8888` (Unsloth Studio) e mostra il selettore modelli e la vista **Model Arena**.
2. Spiega che un modello AI è come un file immagine o un modello CAD 3D: non compresso pesa 6–15 GB e richiede schede video da 2.000 €; quantizzato in **GGUF (4-bit o 1.58-bit/2-bit)** pesa poche centinaia di MB e gira sulla CPU della VM.
3. Lancia lo stesso prompt tecnico nella Model Arena confrontando `Llama-3.2-1B (UD-IQ2_M)` e `Qwen2.5-Coder-1.5B (Q4_K_M)`.

- **Studenti sulla VM:**

1. Aprono **Model Arena** su Unsloth Studio e selezionano i due modelli pre-caricati.
2. Incollano questo **Prompt di Test Industriale**:

```
    Sei l'assistente del reparto manutenzione. Leggi questa nota operatore:
```

```
    "Ore 14:20 - Pressa P03 ferma per sovratemperatura olio idraulico (misurati 78°C, limite massimo 65°C). Sostituito filtro intasato cod. FLT-992, fermo macchina totale 45 minuti."

    Estrai una tabella con: Macchina, Causa Guasto, Temperatura Rilevata, Delta superamento limite (°C), Ricambio Usato, Minuti Fermo.
```

1. Osservano:

- Quanti **token/secondo** fa il modello più piccolo rispetto al più grande?
- Il modello ultra-compresso (2-bit) ha calcolato giusto il Delta (`78 - 65 = 13°C`) o ha inventato il numero?

1. Scelgono `Qwen2.5-Coder-1.5B-Instruct-GGUF (Q4_K_M)` e lo lasciano caricato come motore attivo per il blocco successivo.

5.4 I 5 Problemi più Frequenti in Aula e Soluzioni Rapide ("Pronto Soccorso Docente")

#	Problema in Aula (Sintomo Visibile)	Causa Tecnica Reale	Soluzione Rapida in Aula (< 60 secondi)
1	**OpenCode risponde in chat con il codice, ma NON crea né modifica il file nella cartella**	Unsloth è stato avviato con `unsloth run` senza `--disable-tools` (Unsloth cattura le tool-call invece di passarle a OpenCode), oppure il modello caricato è troppo piccolo (< 1B) e fallisce il formato tool-call.	1) Chiudere e rilanciare tramite `unsloth start opencode` (che configura il passthrough), oppure se si usa `unsloth run` aggiungere **sempre** `--disable-tools`. 2) Se il modello 0.6B/1B scrive il codice in chat, usare il pulsante *Copy* o passare a `Qwen2.5-Coder-1.5B`.
2	**La VM diventa lentissima o la ventola del server esplode quando tutti generano il codice v2**	24 istanze di `llama.cpp` dentro Unsloth Studio stanno usando tutti i thread delle vCPU contemporaneamente sullo stesso host VMware.	**Passaggio al Piano B (LAN Docente):** Gli studenti fermano il modello locale su Unsloth Studio (pulsante *Unload/Stop*) e puntano OpenCode all'indirizzo IP del PC docente (`http://192.168.x.100:8888/v1/`) o al Piano C (API key gratuita).
3	**AI passo v3 l'AI "impazzisce": cercando di aggiungere una funzione cancella metà della pagina o i pulsanti non cliccano più**	I modelli piccoli (1.5B–3B), quando riscrivono un file lungo più di 250 righe, possono tagliare parti di codice esistente o dimenticare di chiudere un tag `<script>`.	**Momento didattico d'oro!** Non perdere 20 minuti a cercare la parentesi mancante: 1) Eseguire `git restore .` per tornare subito alla `v2` funzionante. 2) Riformulare il prompt in modo più corto e specifico: *"Modifica SOLO la funzione di esportazione senza toccare il resto della pagina"*.
4	**Git blocca lo studente con l'errore `Author identity unknown` (`Please tell me who you are`) al primo commit**	Sulla VM non sono stati configurati `user.name` e `user.email` prima del primo `git commit`.	Far eseguire i due comandi (o inserirli già nella Golden Image): `git config --global user.name "Studente Maker"` e `git config --global user.email "studente@itsmaker.it"`

#	Problema in Aula (Sintomo Visibile)	Causa Tecnica Reale	Soluzione Rapida in Aula (< 60 secondi)
5	**Unsloth Studio segna `OOM` (Out of Memory) o si chiude da solo mentre carica il modello**	Lo studente ha aperto contemporaneamente 15 schede di Chrome, VS Code e ha provato a caricare due modelli da 3B nella Model Arena su una VM con soli 6 GB di RAM.	1) Chiudere le schede browser inutili. 2) Usare un solo modello alla volta nella scheda **Chat** (non Arena) durante il Vibe Coding. 3) Rimanere sui tagli `0.6B` – `1.5B Q4_K_M` (~1 GB di RAM).

---

6. Prossimi Passi Operativi per lo Sviluppo Materiali

1. **Blocco 1 – Setup Tecnico VMware & Script "Golden Image":** generazione degli script automatici (`.sh` / `.ps1`) per pre-scaricare i modelli GGUF, creare i repository locali di prova e configurare il server di backup LAN del docente.
2. **Blocco 2 – Scelta e Sviluppo Completo del Progetto di Vibe Coding:** generazione del codice sorgente di partenza, dei file CSV "sporchi" per l'ufficio tecnico e delle soluzioni `v1` / `v2` / `v3` per il docente.
3. **Blocco 3 – Dettaglio Esercizi, Slide Docente e Scheda Operativa Studenti:** dispensa passo-passo stampabile/PDF per i ragazzi, griglia del Banco Prova GGUF da compilare e checklist di autovalutazione.